1. Install libraries

```
!pip install -q -U pypdf pandas google-genai cohere groq huggingface_hub
```

2. Import Libraries

```
import os
import re
import json
import zipfile
from pathlib import Path
from getpass import getpass

import pandas as pd
from pypdf import PdfReader

from google.colab import files

from google import genai
import cohere
from groq import Groq
from huggingface_hub import InferenceClient
```

3. upload zip file

```
uploaded = files.upload()
```

Choose Files FTSP (Week 1) .zip
FTSP (Week 1) .zip(application/x-zip-compressed) - 89575 bytes, last modified: 3/9/2026 - 100% done

```
Saving FTSP (Week 1)  .zip to FTSP (Week 1)   (1).zip
```

4. extract zip file

```
extract_folder = Path("project_files")
extract_folder.mkdir(exist_ok=True)

zip_filename = list(uploaded.keys())[0]

with zipfile.ZipFile(zip_filename, "r") as zip_ref:
    zip_ref.extractall(extract_folder)

print("Files extracted successfully.\n")

for file in sorted(extract_folder.iterdir()):
    print(file.name)
```

```
Files extracted successfully.

1 DV.txt
2 DV.txt
3.txt
4.txt
5.txt
Sr. Staff Engineer, Design Verification.pdf
```

5. Locate resumes and job description

```
txt_files = sorted(extract_folder.glob("*.txt"))
pdf_files = sorted(extract_folder.glob("*.pdf"))

print("Resume files found:")
for f in txt_files:
    print("-", f.name)

print("\nPDF files found:")
for f in pdf_files:
    print("-", f.name)
```

```
if len(pdf_files) == 0:
    raise FileNotFoundError("No PDF job description found in the extracted folder.")
```

```
Resume files found:
- 1 DV.txt
- 2 DV.txt
- 3.txt
- 4.txt
- 5.txt

PDF files found:
- Sr. Staff Engineer, Design Verification.pdf
```

6. Read all text files

```
def read_text_file(file_path):
    with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
        return f.read()

raw_resumes = {}
for file_path in txt_files:
    raw_resumes[file_path.name] = read_text_file(file_path)

print("Loaded resumes:", len(raw_resumes))
print("\nSample preview:\n")

for name, text in raw_resumes.items():
    print("=" * 80)
    print(name)
    print(text[:500])
    print()
```

```
Top Skills
AXI
APB
CHI
Languages
English  (Native or Bilingual)
Mandarin  (Limited Working)
Malay  (Native or Bilingual)
Design Verification Engineer
Kulim, Kedah, Malaysia
Summary
Currently working as Design Verification Engineer at Skyechip Sdn
Bhd
Graduated from University of Malaysia Perlis (UniMAP), majoring in
Microelectronic Engineering.
Experience
SkyeChip
Design Verification Engineer
August 2022-Present (1 year 4 months)
Penang, Malaysia
Fuji Electric (M) Sdn Bhd
Manufacturing Engin

================================================================================
2 DV.txt

Top Skills
RTL Coding
RTL Verification
RTL Development
Languages
English
Staff Design Verification
Ho Chi Minh City Metropolitan Area
Experience
BOS Semiconductors
Staff Design Verification Engineer
December 2022-Present (1 year)
HCL Technologies
Technical Lead
March 2022-December 2022 (10 months)
```

```
3.txt

Principales comptences
C++
Verilog
FPGA
Languages
English  (Limited Working)
Design Verification Leader
Vit Nam
```

7. Read the job description PDF

```
def read_pdf_text(pdf_path):
    reader = PdfReader(str(pdf_path))
    text = []
    for page in reader.pages:
        page_text = page.extract_text()
        if page_text:
            text.append(page_text)
    return "\n".join(text)

job_pdf_path = pdf_files[0]
raw_job_description = read_pdf_text(job_pdf_path)

print("Job description file:", job_pdf_path.name)
print("\nPreview:\n")
print(raw_job_description[:1500])
```

```
Job description file: Sr. Staff Engineer, Design Verification.pdf

Preview:

Sr. Staff Engineer, Design Verification

Staff Engineer – Design Verification

Client is looking for an experienced Staff Design Verification Engineer to join our dynamic team in
Austin, TX. The ideal candidate will have 8-12 years of experience in the field of semiconductor
design verification, and a proven track record of success in developing and executing verification
plans for complex SoC designs.

Responsibilities:

The Design Verification Engineer will be responsible for verification of digital and mixed-signal
designs, including systems-on-chip with multiple CPUs, and digital signal processors, security
hardware, and other logic for IoT applications.

Specific Responsibilities:
• Ideal candidate should have demonstrated successful design verification tasks at block, sub-
system, and full-chip level.
• Must have participated in all phases of chip development, from creating test plans, creating
testbench environment (SV/UVM), integrate VIP’s, automate test env for randomized testing
and score boarding.
• Utilize UVM to create drivers, monitors, predictors, and scoreboards.
• In-Depth knowledge of SoC architecture with AMBA AXI/AHB/APB, DMA’s, Security, clock,
and power-gating techniques is required.
• Responsible for verification of block(s) that includes writing tests, assertion and coverage for
a block.
• Create tests to achieve coverage goals while verifying functionality.
• Develop support utilities for verification automation, test bench a
```

8. Text preprocessing

```
def preprocess_text(text):
    if not isinstance(text, str):
        text = str(text)

    # normalize line breaks / spaces
    text = text.replace("\xa0", " ")
    text = text.replace("\n", " ")
    text = text.replace("\t", " ")

    # lowercase
    text = text.lower()

    # remove phone numbers
```

```
    text = re.sub(r'(\+?\d[\d\-\(\) ]{7,}\d)', ' ', text)

    # remove repeated punctuation-like noise but keep useful technical symbols
    # keep letters, digits, spaces, slash, plus, hash, hyphen, dot
    text = re.sub(r'[^a-z0-9/\+\#\-\.\s]', ' ', text)

    # remove multiple dots or dashes if noisy
    text = re.sub(r'[\.]{2,}', ' ', text)
    text = re.sub(r'[-]{2,}', ' ', text)

    # collapse spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text
```

9. Apply preprocessing to resumes and job description

```
cleaned_resumes = {}
for name, text in raw_resumes.items():
    cleaned_resumes[name] = preprocess_text(text)

cleaned_job_description = preprocess_text(raw_job_description)

print("Cleaned job description preview:\n")
print(cleaned_job_description[:1200])

print("\n\nCleaned resume preview:\n")
for name, text in cleaned_resumes.items():
    print("=" * 80)
    print(name)
    print(text[:400])
    print()
```

```
Cleaned job description preview:

sr. staff engineer design verification staff engineer design verification client is looking for an experienced staff design ve


Cleaned resume preview:

================================================================================
1 DV.txt
top skills axi apb chi languages english native or bilingual mandarin limited working malay native or bilingual design verific

================================================================================
2 DV.txt
top skills rtl coding rtl verification rtl development languages english staff design verification ho chi minh city metropolit

================================================================================
3.txt
principales comptences c++ verilog fpga languages english limited working design verification leader vit nam exprience coasia

================================================================================
4.txt
top skills c c++ java languages malay professional working english full professional chinese simplified native or bilingual ho

================================================================================
5.txt
top skills very-large-scale integration vlsi system on a chip soc innovus physical design engineer with various experiences in
```

10. Save cleaned text files

```
cleaned_folder = Path("cleaned_texts")
cleaned_folder.mkdir(exist_ok=True)

for name, text in cleaned_resumes.items():
    out_path = cleaned_folder / f"cleaned_{name}"
    with open(out_path, "w", encoding="utf-8") as f:
        f.write(text)

with open(cleaned_folder / "cleaned_job_description.txt", "w", encoding="utf-8") as f:
    f.write(cleaned_job_description)

print("Cleaned files saved in:", cleaned_folder)
```

```
print("\nSaved files:")
for f in sorted(cleaned_folder.iterdir()):
    print("-", f.name)
```

```
Cleaned files saved in: cleaned_texts

Saved files:
- cleaned_1 DV.txt
- cleaned_2 DV.txt
- cleaned_3.txt
- cleaned_4.txt
- cleaned_5.txt
- cleaned_job_description.txt
```

11. preprocessing summary table

```
summary_rows = []

for name in raw_resumes:
    raw_text = raw_resumes[name]
    cleaned_text = cleaned_resumes[name]

    summary_rows.append({
        "file_name": name,
        "raw_char_count": len(raw_text),
        "cleaned_char_count": len(cleaned_text),
        "removed_chars": len(raw_text) - len(cleaned_text)
    })

summary_rows.append({
    "file_name": "job_description_pdf",
    "raw_char_count": len(raw_job_description),
    "cleaned_char_count": len(cleaned_job_description),
    "removed_chars": len(raw_job_description) - len(cleaned_job_description)
})

summary_df = pd.DataFrame(summary_rows)
summary_df
```

	file_name	raw_char_count	cleaned_char_count	removed_chars
0	1 DV.txt	2974	2883	91
1	2 DV.txt	1403	1362	41
2	3.txt	1699	1646	53
3	4.txt	4800	4692	108
4	5.txt	4376	4215	161
5	job_description_pdf	2562	2422	140

12. Create prompts

```
SYSTEM_PROMPT = """
You are an HR screening assistant.
Your task is to compare a candidate's resume with a job description.

Focus on:
1. Relevant technical skills
2. Relevant work experience
3. Overall suitability for the role

Return the answer in a clear and concise format with:
- Candidate Summary
- Matching Skills
- Missing / Weak Areas
- Suitability Score out of 10
"""
```

```
def build_user_prompt(job_description, candidate_name, resume_text):
    return f"""
Job Description:
{job_description}
```

```
Candidate Resume File Name:
{candidate_name}

Candidate Resume Text:
{resume_text}

Please evaluate this candidate against the job description.

Format your answer as:
Candidate Summary:
Matching Skills:
Missing / Weak Areas:
Suitability Score:
"""
```

13. Enter API keys

```
COHERE_API_KEY = getpass("Enter your Cohere API key: ")
GEMINI_API_KEY = getpass("Enter your Gemini API key: ")
GROQ_API_KEY = getpass("Enter your Groq API key: ")
HF_API_KEY = getpass("Enter your Hugging Face API key: ")
```

```
Enter your Cohere API key: ··********
Enter your Gemini API key: ··********
Enter your Groq API key: ··********
Enter your Hugging Face API key: ··********
```

14. Set up API clients

```
# Gemini client
gemini_client = genai.Client(api_key=GEMINI_API_KEY)

# Cohere client
co = cohere.ClientV2(api_key=COHERE_API_KEY)

# Groq client
groq_client = Groq(api_key=GROQ_API_KEY)

# Hugging Face client
hf_client = InferenceClient(api_key=HF_API_KEY)
```

15. Gemini response function

```
def get_gemini_response(system_prompt, user_prompt, model_name="gemini-2.5-flash"):
    full_prompt = f"System Prompt:\n{system_prompt}\n\nUser Prompt:\n{user_prompt}"

    response = gemini_client.models.generate_content(
        model=model_name,
        contents=full_prompt
    )

    return response.text
```

16. Cohere response function

```
def get_cohere_response(system_prompt, user_prompt, model_name="command-r-08-2024"):
    response = co.chat(
        model=model_name,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ]
    )

    # Cohere V2 responses may return content blocks
    if hasattr(response, "message") and hasattr(response.message, "content"):
        parts = []
        for item in response.message.content:
            if hasattr(item, "text"):
                parts.append(item.text)
```

```
            elif isinstance(item, dict) and "text" in item:
                parts.append(item["text"])
        return "\n".join(parts).strip()

    return str(response)
```

17. Grok Function

```
def get_groq_response(system_prompt, user_prompt, model_name="llama-3.3-70b-versatile"):
    response = groq_client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ]
    )
    return response.choices[0].message.content
```

18. HuggingFace Function

```
def get_huggingface_response(
    system_prompt,
    user_prompt,
    model_name="meta-llama/Llama-3.1-70B-Instruct"
):
    response = hf_client.chat.completions.create(
        model=model_name,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        max_tokens=800
    )
    return response.choices[0].message.content
```

19. Test one resume first

```
test_candidate = list(cleaned_resumes.keys())[0]
test_resume = cleaned_resumes[test_candidate]

test_prompt = build_user_prompt(
    job_description=cleaned_job_description,
    candidate_name=test_candidate,
    resume_text=test_resume
)

print("Testing candidate:", test_candidate)

print("\n--- GEMINI OUTPUT ---\n")
try:
    print(get_gemini_response(SYSTEM_PROMPT, test_prompt))
except Exception as e:
    print("Gemini Error:", e)

print("\n--- COHERE OUTPUT ---\n")
try:
    print(get_cohere_response(SYSTEM_PROMPT, test_prompt))
except Exception as e:
    print("Cohere Error:", e)

print("\n--- GROQ OUTPUT ---\n")
try:
    print(get_groq_response(SYSTEM_PROMPT, test_prompt))
except Exception as e:
    print("Groq Error:", e)

print("\n--- HUGGING FACE OUTPUT ---\n")
try:
    print(get_huggingface_response(SYSTEM_PROMPT, test_prompt))
except Exception as e:
    print("Hugging Face Error:", e)
```

```
Candidate Summary:
This candidate is a Design Verification Engineer with a background in microelectronic engineering. They have experience in th

Matching Skills:
- Experience in Design Verification: The candidate has worked as a Design Verification Engineer, which aligns with the job's
- Understanding of Low-Power Design: Their experience with low-power design and verification challenges is a strong match for
- Knowledge of Languages and Technologies: Proficiency in languages such as SystemVerilog, UVM, Verilog, C/C++, and Python is

Missing / Weak Areas:
- Years of Experience: The job description requires 8-12 years of experience, while the candidate's resume indicates a shorte
- Specific Verification Tasks: The resume does not explicitly mention tasks such as test plan creation, testbench environment
- Architecture Knowledge: While the candidate has experience with AMBA AXI/APB, the resume does not highlight in-depth knowle

Suitability Score: 6/10

The candidate's skills and experience show a good foundation for the role, particularly in design verification and low-power

--- GROQ OUTPUT ---

Candidate Summary:
The candidate is a Design Verification Engineer with approximately 1 year of experience in design verification and a backgrou

Matching Skills:
- Experience with AXI and APB protocols
- Background in microelectronic engineering
- Proficiency in English language
- Familiarity with design verification concepts

Missing / Weak Areas:
- Lack of experience in SystemVerilog, UVM, and Verilog
- Limited experience in SOC architecture, low-power design, and verification challenges
- No direct experience with ARM processor-based designs, low-power design techniques, or MIPS
- Limited experience in C-based verification and test automation
- No mention of experience with gate-level simulations, FPGA, or post-silicon bring-up

Suitability Score: 4/10
The candidate has some basic skills that match the job requirements, but they lack significant experience and technical skill

--- HUGGING FACE OUTPUT ---

Candidate Summary:
The candidate is a Design Verification Engineer with 1 year 4 months of experience in the field, currently working at Skyechi

Matching Skills:
- The candidate has experience with AXI and APB, which are relevant to the job description.
- They have a degree in Microelectronic Engineering, which is relevant to the field of semiconductor design verification.
- They have worked as a Design Verification Engineer, which suggests they have some experience in the field.

Missing / Weak Areas:
- The candidate lacks experience in SystemVerilog, UVM, and Verilog, which are essential skills for the job.
- They have limited experience in design verification, with only 1 year 4 months of experience in the field.
- They do not have experience with SOC architecture, AMBA, DMA, or low-power design techniques, which are required for the jo
- They do not have experience with ARM processor-based designs, which is a preferred technology for the job.
- They lack experience in working with mixed-signal designs, low-power battery-operated designs, and design verification for

Suitability Score: 2/10
The candidate's limited experience in design verification, lack of essential skills, and limited relevance of their work hist
```

20. Run all resumes through models

```
results = []

for candidate_name, resume_text in cleaned_resumes.items():
    user_prompt = build_user_prompt(
        job_description=cleaned_job_description,
        candidate_name=candidate_name,
        resume_text=resume_text
    )

    print(f"Processing {candidate_name}...")

    try:
        gemini_output = get_gemini_response(SYSTEM_PROMPT, user_prompt)
    except Exception as e:
        gemini_output = f"Gemini Error: {e}"

    try:
        cohere_output = get_cohere_response(SYSTEM_PROMPT, user_prompt)
    except Exception as e:
```

```
            cohere_output = f"Cohere Error: {e}"

        try:
            groq_output = get_groq_response(SYSTEM_PROMPT, user_prompt)
        except Exception as e:
            groq_output = f"Groq Error: {e}"

        try:
            huggingface_output = get_huggingface_response(SYSTEM_PROMPT, user_prompt)
        except Exception as e:
            huggingface_output = f"Hugging Face Error: {e}"

        results.append({
            "candidate_name": candidate_name,
            "gemini_output": gemini_output,
            "cohere_output": cohere_output,
            "groq_output": groq_output,
            "huggingface_output": huggingface_output
        })

    print("Done.")
```

```
Processing 1 DV.txt...
Processing 2 DV.txt...
Processing 3.txt...
Processing 4.txt...
Processing 5.txt...
Done.
```

21. View all results

```
for item in results:
    print("=" * 100)
    print("Candidate:", item["candidate_name"])

    print("\nGEMINI RESPONSE:\n")
    print(item["gemini_output"])

    print("\nCOHERE RESPONSE:\n")
    print(item["cohere_output"])

    print("\nGROQ RESPONSE:\n")
    print(item["groq_output"])

    print("\nHUGGING FACE RESPONSE:\n")
    print(item["huggingface_output"])
    print()
```

```
- No clear evidence of experience with C-based verification in an SOC environment

Suitability Score: 4/10
The candidate has some relevant experience in digital design and physical verification, but lacks experience in design verifi

HUGGING FACE RESPONSE:

Candidate Summary:
The candidate is an experienced semiconductor engineer with a background in backend digital design, having worked at Intel ar

Matching Skills:
- Experience with SOC designs and architectures
- Knowledge of clock distribution and timing verification
- Familiarity with digital design and backend processes
- Strong understanding of physical design and implementation
- Experience with scripting and automation (custom scripts)

Missing / Weak Areas:
- Lack of direct experience in design verification, particularly at the block, sub-system, and full-chip levels
- Limited experience with SystemVerilog, UVM, and verification methodologies
- No clear experience with mixed-signal designs, low-power design, and verification challenges
- No mention of experience with ARM processor-based designs, AMBA, AXI/AHB/APB, DMA, or other relevant technologies
- Limited experience with verification automation, testbench automation, and regression testing

Suitability Score: 4/10
The candidate's background in physical design and clock architecture is notable, but they lack the direct experience and skil
```

Recomendation Section

22. Score Extraction

```
def extract_score(text):
    """
    Extracts a score out of 10 from model output.
    Handles cases like:
    7/10
    7.5/10
    Suitability Score: 6/10
    """
    if not isinstance(text, str):
        return None

    match = re.search(r'(\d+(?:\.\d+)?)\s*/\s*10', text)
    if match:
        return float(match.group(1))

    return None
```

23. Score Table

```
score_rows = []

for item in results:
    score_rows.append({
        "candidate_name": item["candidate_name"],
        "gemini_score": extract_score(item["gemini_output"]),
        "cohere_score": extract_score(item["cohere_output"]),
        "groq_score": extract_score(item["groq_output"]),
        "huggingface_score": extract_score(item["huggingface_output"]),
    })

scores_df = pd.DataFrame(score_rows)
scores_df
```

	candidate_name	gemini_score	cohere_score	groq_score	huggingface_score
0	1 DV.txt	2.0	6.0	4.0	2.0
1	2 DV.txt	5.0	7.0	8.0	6.5
2	3.txt	8.5	6.0	7.5	7.5
3	4.txt	3.0	5.0	NaN	4.0
4	5.txt	2.0	6.0	4.0	4.0

24. Average Score

```
score_columns = ["gemini_score", "cohere_score", "groq_score", "huggingface_score"]

scores_df["average_score"] = scores_df[score_columns].mean(axis=1, skipna=True)
scores_df = scores_df.sort_values(by="average_score", ascending=False)

scores_df
```

	candidate_name	gemini_score	cohere_score	groq_score	huggingface_score	average_score
2	3.txt	8.5	6.0	7.5	7.5	7.375
1	2 DV.txt	5.0	7.0	8.0	6.5	6.625
3	4.txt	3.0	5.0	NaN	4.0	4.000
4	5.txt	2.0	6.0	4.0	4.0	4.000
0	1 DV.txt	2.0	6.0	4.0	2.0	3.500

ALL COMBINED RESULTS

```
from collections import Counter

score_columns = ["gemini_score", "cohere_score", "groq_score", "huggingface_score"]

# Find top candidate for each model
model_recommendations = {}

for col in score_columns:
    valid_df = scores_df.dropna(subset=[col])
    if not valid_df.empty:
        top_candidate = valid_df.sort_values(by=col, ascending=False).iloc[0]["candidate_name"]
        model_recommendations[col] = top_candidate

print("Top recommendation by each LLM:\n")
for model_name, candidate in model_recommendations.items():
    print(f"{model_name}: {candidate}")

# Majority vote
votes = list(model_recommendations.values())
vote_counter = Counter(votes)

print("\nMajority vote result:\n")
for candidate, count in vote_counter.items():
    print(f"{candidate}: {count} vote(s)")

majority_best_candidate = vote_counter.most_common(1)[0][0]

# Best candidate by average score
best_average_candidate = scores_df.sort_values(by="average_score", ascending=False).iloc[0]["candidate_name"]

print("\nBest resume based on average score:", best_average_candidate)

# Final decision
if majority_best_candidate == best_average_candidate:
    final_best_candidate = majority_best_candidate
else:
    final_best_candidate = best_average_candidate

print("\nFINAL RECOMMENDED RESUME:", final_best_candidate)
```

```
Top recommendation by each LLM:

gemini_score: 3.txt
cohere_score: 2 DV.txt
groq_score: 2 DV.txt
huggingface_score: 3.txt

Majority vote result:

3.txt: 2 vote(s)
2 DV.txt: 2 vote(s)

Best resume based on average score: 3.txt

FINAL RECOMMENDED RESUME: 3.txt
```

preprcessing testing

```
candidate = list(raw_resumes.keys())[0]

print("RAW TEXT:\n")
print(raw_resumes[candidate][:500])

print("\n" + "="*60 + "\n")

print("CLEANED TEXT:\n")
print(cleaned_resumes[candidate][:500])
```

```
RAW TEXT:


Top Skills
AXI
APB
CHI
Languages
English  (Native or Bilingual)
Mandarin  (Limited Working)
Malay  (Native or Bilingual)
Design Verification Engineer
Kulim, Kedah, Malaysia
Summary
Currently working as Design Verification Engineer at Skyechip Sdn
Bhd
Graduated from University of Malaysia Perlis (UniMAP), majoring in
Microelectronic Engineering.
Experience
SkyeChip
Design Verification Engineer
August 2022-Present (1 year 4 months)
Penang, Malaysia
Fuji Electric (M) Sdn Bhd
Manufacturing Engin

============================================================

CLEANED TEXT:

top skills axi apb chi languages english native or bilingual mandarin limited working malay native or bilingual design verific
```

```
for name in raw_resumes:
    raw_len = len(raw_resumes[name])
    clean_len = len(cleaned_resumes[name])

    print(name)
    print("Raw length:", raw_len)
    print("Cleaned length:", clean_len)
    print("Removed characters:", raw_len - clean_len)
    print()
```

```
1 DV.txt
Raw length: 2974
Cleaned length: 2883
Removed characters: 91

2 DV.txt
Raw length: 1403
Cleaned length: 1362
```

```
Removed characters: 41

3.txt
Raw length: 1699
Cleaned length: 1646
Removed characters: 53

4.txt
Raw length: 4800
Cleaned length: 4692
Removed characters: 108

5.txt
Raw length: 4376
Cleaned length: 4215
Removed characters: 161
```

```
import re

for name, text in cleaned_resumes.items():
    emails = re.findall(r'\S+@\S+', text)

    if emails:
        print(name, "Still has email:", emails)
    else:
        print(name, "No emails found")
```

```
1 DV.txt No emails found
2 DV.txt No emails found
3.txt No emails found
4.txt No emails found
5.txt No emails found
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.